LORA

LoRA (Low-Rank Adaptation) is a Parameter-Efficient Fine Tuning method that adapts large pretrained transformer models by freezing original weights and inserting small trainable low-rank matrices into important linear layers.

Major Problems of Full Fine-Tuning Solved by LoRA are -

Problem 1: High GPU Memory Usage

Full FT needs 3–4× memory (params + gradients + optimizer states), making it infeasible on consumer GPUs, while LoRA trains <1% parameters → huge memory reduction.

Problem 2: Slow & Expensive Training

Full FT updates all layers requiring heavy computation and long training time, while LoRA trains fewer parameters → faster training + lower cost.

Problem 3: Storage & Deployment Explosion

Full FT requires storing separate full models (GBs) per task, while LoRA stores small adapters (MBs) → efficient multi-task deployment.

Bonus: Knowledge Preservation

Full FT causes catastrophic forgetting by overwriting pretrained weights, while LoRA freezes base model → preserves knowledge.

Training Flow of LoRA -

Step 1 — Load Pretrained Transformer

Start with a pretrained model having strong language understanding and general knowledge.

Step 2 — Freeze Base Parameters

All original weights are frozen (no updates) → preserves knowledge + reduces memory.

Step 3 — Inject LoRA Modules

Add low-rank adapters to selected linear layers → creates lightweight trainable paths.

Step 4 — Forward Pass

Output = Original output + LoRA correction → modifies behavior efficiently.

Step 5 — Loss Computation

Model predictions compared with ground truth using task-specific loss.

Step 6 — Backpropagation

Gradients flow only through LoRA parameters → base model remains unchanged.

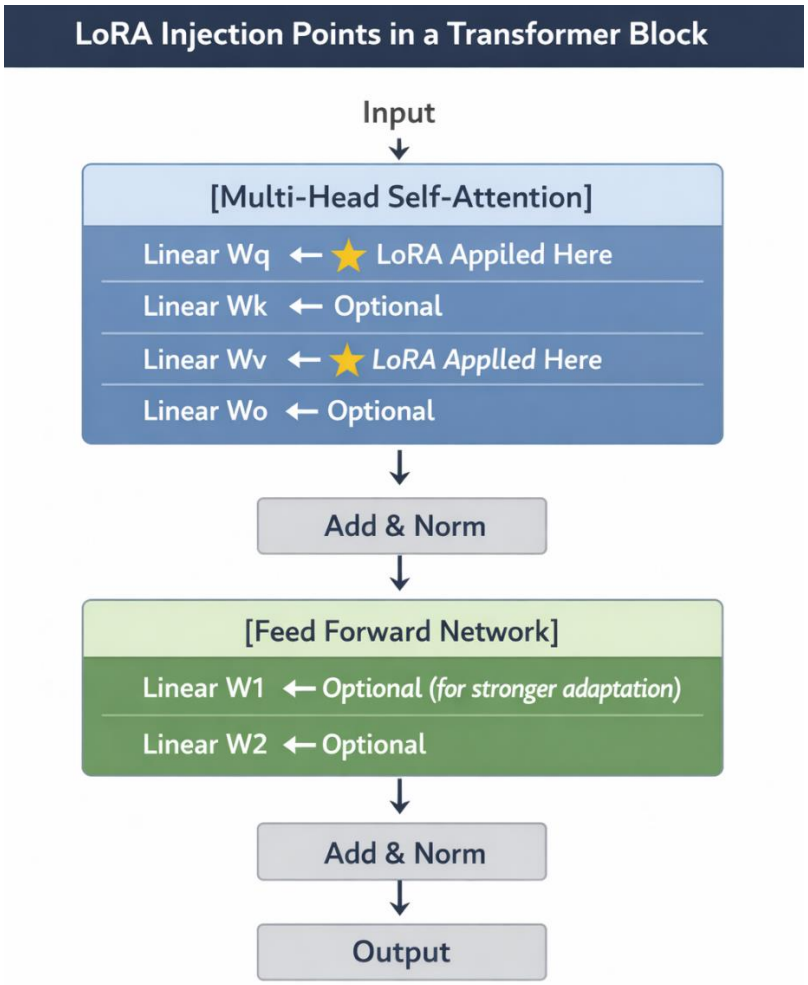
Step 7 — Parameter Update

Optimizer updates only LoRA matrices → fast and memory-efficient training.

Step 8 — Save Adapters

Store only small LoRA adapters → base model reused for multiple tasks.

Exact Locations Where LoRA Is Applied in Transformers –



Core Weight Update Rule (LoRA)

$$W' = W + \frac{\alpha}{r} BA$$

Practical Meaning

- W = frozen pretrained weight matrix
- A, B = small trainable low-rank matrices
- r = rank → controls adapter capacity
- α = scaling factor → controls update strength
- LoRA adds a learned correction (BA), not modifying W directly

Rank Selection Strategy

Rank (r) controls LoRA adapter capacity.

Low r = efficient but limited learning, High r = better learning but more memory.

◈ Practical Rank Guidelines

- r = 2–4 → small tasks (classification, low data)
- r = 8 → standard NLP tasks (default)
- r = 16 → complex generation tasks
- r = 32+ → heavy domain shift (medical/legal)

How Engineers Choose Rank

Start with low r → monitor validation → increase if underfitting.

Rule of Thumb

Small dataset + high rank → overfitting → match rank with task complexity.

LoRA Hyperparameters

- **Rank (r)** - Controls adapter capacity → low = underfitting, high = overfitting + memory waste
- **Alpha (α)** - Controls update strength → higher α = stronger adaptation (common: α ≈ 2r)
- **Target Modules** - Decide where LoRA is applied → typically Wq, Wv (FFN for stronger adaptation)
- **Dropout** - Regularizes adapters → prevents overfitting (typical: 0.05–0.1)
- **Learning Rate** - Training speed control → higher than full FT (typical: 1e-4 to 5e-4)

QLORA

QLoRA (Quantized Low-Rank Adaptation) is an advanced Parameter-Efficient Fine-Tuning technique that combines 4-bit weight quantization with LoRA adapters to enable training extremely large language models using very limited GPU memory.

QLoRA : Problems & Solutions

- **GPU Memory Limits** - Large LLMs need huge VRAM making fine-tuning infeasible, while QLoRA uses 4-bit quantization → ~75% memory reduction.
- **High Training Cost** - Full precision training requires heavy computation and optimizer states, while QLoRA uses frozen quantized weights + LoRA → faster and cheaper training.
- **Hardware Accessibility** - Large models require multi-GPU clusters, while QLoRA enables fine-tuning 30B–65B models on a single consumer GPU.
- **Accuracy Preservation** - Quantization may reduce performance, while QLoRA uses careful techniques → near full-precision accuracy.

QLoRA = Quantization + LoRA → low memory + fast training + single GPU fine-tuning

QLoRA Training Flow

- **Step 1** — Load Pretrained Model - Start with pretrained model in standard precision
- **Step 2 —** Quantize Weights - Convert weights to 4-bit → compress memory while preserving information
- **Step 3 —** Freeze Quantized Weights - Base model remains fixed → acts as knowledge backbone
- **Step 4** — Inject LoRA Adapters - Add low-rank matrices to selected layers → enable task learning
- **Step 5 —** Forward Pass - Quantized output + LoRA correction → modifies behavior efficiently
- **Step 6 —** Loss Computation - Predictions compared with labels using task-specific loss
- **Step 7 —** Backpropagation - Gradients update only LoRA → base weights unchanged
- **Step 8** — Save Adapters - Store quantized model + small LoRA adapters for deployment

Key Difference from LoRA

The base linear layers use quantized weights instead of full-precision weights. LoRA adapters remain full precision for stable learning.

Mathematical formulation –

QLoRA Weight Update Rule

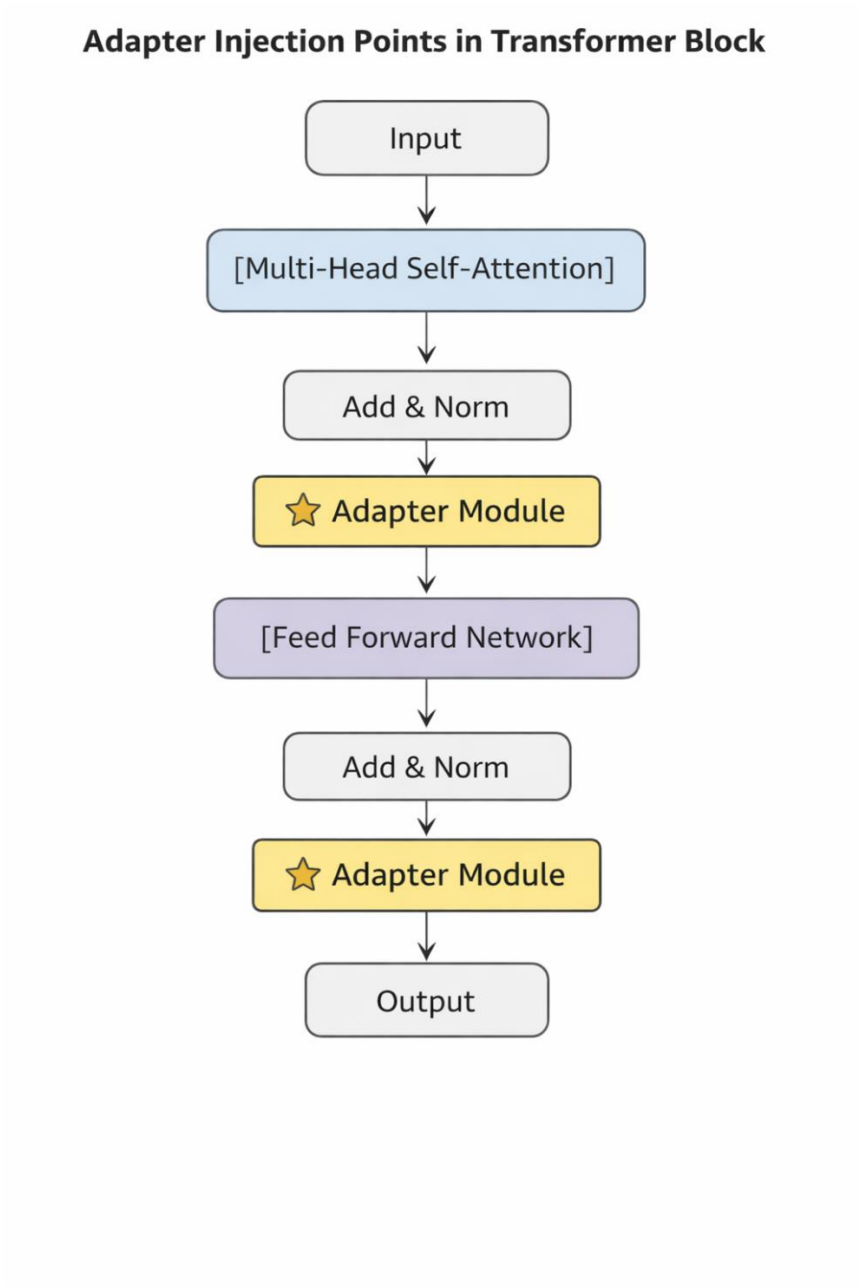
$$W' = Q(W) + \frac{\alpha}{r} BA$$

Practical Meaning

- Q(W) - Frozen quantized pretrained weights (4-bit)
- A, B - Small trainable low-rank matrices
- r - Rank → controls adapter capacity
- α - Scaling factor → controls update strength
- Only A and B are trained → base quantized weights remain unchanged

ADAPTERS

Adapters are Parameter-Efficient Fine-Tuning modules where small trainable neural networks are inserted inside transformer layers to adapt pretrained models for new tasks while keeping original weights frozen.



Adapter Training Flow

- **Step 1 — Load Pretrained Transformer** - Start with pretrained model having general language intelligence
- **Step 2 — Freeze Base Weights** - All parameters frozen → preserves knowledge + reduces memory
- **Step 3 — Insert Adapter Modules** - Add small bottleneck networks inside transformer layers
- **Step 4 — Forward Pass** - Hidden states pass through adapters → refine task-specific representations

- **Step 5 — Loss Computation** - Predictions compared with labels using task-specific objective
- **Step 6 — Backpropagation** - Gradients update only adapters → base model unchanged
- **Step 7 — Parameter Update** - Optimizer updates small adapter networks → fast and efficient
- **Step 8 — Save Adapter Weights** - Store only adapters → reused with base model for inference

DORA

DoRA decomposes weights into magnitude + direction and fine-tunes only direction → better expressiveness and stability than LoRA.

◈ Core Idea

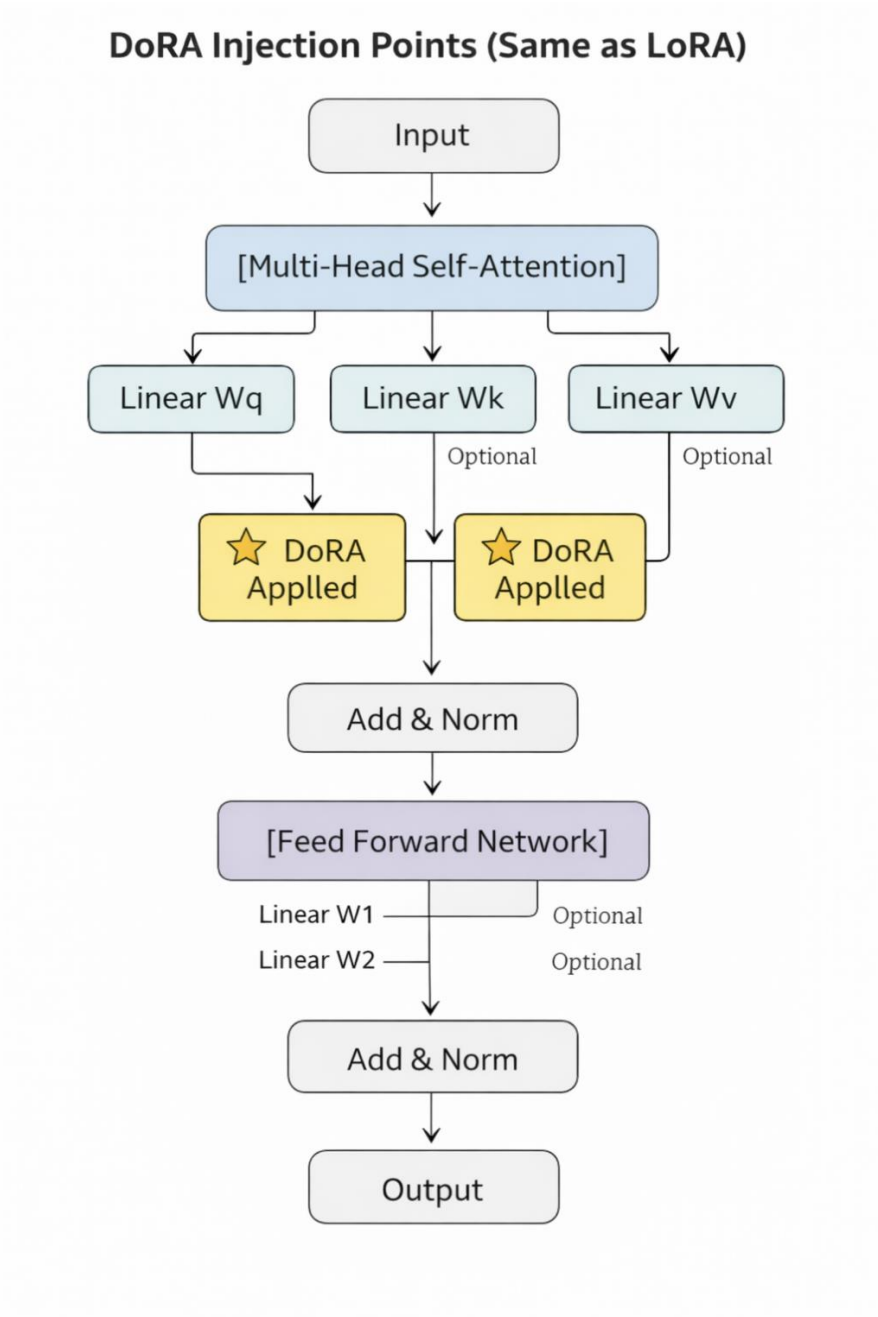
- Magnitude (norm) - Controls feature strength
- Direction - Controls information pattern
- DoRA updates direction only → preserves scale + improves learning capacity

Intuition

DoRA focuses on “where features point” (direction) instead of “how strong they are” (magnitude).

Problems in LoRA → DoRA Solutions

- Limited Expressiveness - LoRA low-rank updates may lack capacity, while DoRA separates magnitude & direction → richer updates
- Training Instability - LoRA scaling can cause unstable updates, while DoRA preserves magnitude → stable gradients
- Suboptimal Adaptation - LoRA additive updates distort pretrained structure, while DoRA updates direction → preserves geometry



DoRA Training Flow

- **Step 1 — Load Pretrained Model** - Start with pretrained weights W
- **Step 2 — Decompose Weights** - Split into magnitude + direction components
- **Step 3 — Freeze Magnitude** - Keep norms fixed → preserve feature strength
- **Step 4 — Adapt Direction** - Apply low-rank updates to direction only
- **Step 5 — Forward Pass** - Recombine magnitude + updated direction
- **Step 6 — Loss Computation** - Compare predictions with labels

- **Step 7 — Backpropagation** - Update only direction adapters
- **Step 8 — Save Adapters** - Store DoRA adapters similar to LoRA

Mathematical Formulation

Weight Decomposition

$$W = m \cdot V$$

Meaning

- m - Magnitude (norm → feature strength)
- V - Direction (unit matrix → structural information)

Direction Update (DoRA)

$$V' = V + \frac{\alpha}{r} BA$$

Meaning

- A, B - Trainable low-rank matrices
- r - Rank → controls capacity
- α - Scaling factor → controls update strength
- Only direction (V) is updated → magnitude (m) remains fixed

Final Reconstructed Weight

After updating the direction the weight matrix is reconstructed = $m \cdot V'$

INSTRUCTION BASED FINE TUNING

IFT trains a pretrained LLM on instruction–response pairs → converts text prediction into task-oriented behavior.

Core Idea

- Pretrained LLM = good text generator but poor task follower
- IFT teaches instruction → response mapping
- Transforms model into assistant-like system

Practical Meaning

- Understands user intent → produces relevant answers
- Generates structured and formatted outputs
- Enables conversational AI systems

Problems → IFT Solutions

- **Poor Instruction Following** - Pretrained models ignore user intent, while IFT learns instruction → response mapping
- **Irrelevant Outputs** - Raw LLMs produce unstructured text, while IFT learns structured answering patterns
- **No Conversational Ability** - Pretrained models lack dialogue skills, while IFT enables multi-turn conversations
- **Multi-Task Limitation** - Single-task behavior, while IFT trains on diverse instructions → multi-task capability

Training Flow

- Step 1 — Load Pretrained LLM - Start with model having language understanding
- Step 2 — Build Dataset - Create instruction–response pairs
- Step 3 — Format Data - Structure as supervised examples
- Step 4 — Supervised Fine-Tuning - Train using teacher forcing
- Step 5 — Loss Computation - Cross-entropy between predicted and true tokens
- Step 6 — Backpropagation - Update model weights
- Step 7 — Behavior Emerges - Model learns assistant-style responses

Where Updates Happen

All major transformer parts updated → attention, FFN, layer norms, output layers (no architecture change).

IFT vs PEFT

- **Purpose** - IFT = teaches behavior, PEFT = adds knowledge efficiently
- **Parameter Updates** - IFT updates most weights, PEFT updates small modules
- **Hardware Cost** - IFT expensive, PEFT efficient
- **Dataset Type** - IFT uses instruction data, PEFT uses task-specific data
- **Architecture** - IFT no change, PEFT adds adapters/LoRA

When Used Together

IFT + LoRA → instruction-following with high efficiency (modern production approach)